Self-Attention & Transformers

Key Concepts & Worked Examples

COSI 115b — Spring 2026

Recitation

Today's Roadmap

1. **Self-attention** — the core mechanism: queries, keys, values
2. **Multi-head attention** — why one head isn't enough
3. **Positional encodings** — why Transformers need them
4. **Decoder architecture** — how GPT-2 puts it all together

These ideas are the foundation of the new assignment, which is up on Moodle now (due April 17).

Self-Attention

The core mechanism

Self-Attention: The Core Equations

Given a query $\boldsymbol{q} \in \mathbb{R}^d$, keys $\{\boldsymbol{k}_1, \ldots, \boldsymbol{k}_n\}$, and values $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$:

Attention Output

$$\boldsymbol{c} = \sum_{i=1}^{n} \boldsymbol{v}_i \, \alpha_i$$

Attention Weights

$$\alpha_i = \frac{\exp(\boldsymbol{k}_i^\top \boldsymbol{q})}{\sum_{j=1}^{n} \exp(\boldsymbol{k}_j^\top \boldsymbol{q})}$$

Key intuition: $\boldsymbol{c}$ is a weighted average of value vectors, where the weights come from how well each key matches the query.

Quick Check: Understanding the Basics

Question 1: What is the range of each α_i? Can α_i ever be exactly 0 or exactly 1?

Quick Check: Understanding the Basics

Question 1: What is the range of each α_i? Can α_i ever be exactly 0 or exactly 1?

Each $\alpha_i \in (0, 1)$, and $\sum_i \alpha_i = 1$. Strictly positive because $\exp(\cdot) > 0$. Never exactly 0 or 1, only approaches them in the limit.

Question 2: What would $\boldsymbol{c}$ look like if all $\alpha_i = 1/n$?

Quick Check: Understanding the Basics

Question 1: What is the range of each α_i? Can α_i ever be exactly 0 or exactly 1?

Each $\alpha_i \in (0, 1)$, and $\sum_i \alpha_i = 1$. Strictly positive because $\exp(\cdot) > 0$. Never exactly 0 or 1, only approaches them in the limit.

Question 2: What would $\boldsymbol{c}$ look like if all $\alpha_i = 1/n$?

$\boldsymbol{c} = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{v}_i$ — the unweighted average of all values. This is the "no preference" case where the query matches all keys equally.

What Does the Dot Product $\boldsymbol{k}_i^\top \boldsymbol{q}$ Tell Us?

Recall the geometric form: $\boldsymbol{k}_i^\top \boldsymbol{q} = \|\boldsymbol{k}_i\| \, \|\boldsymbol{q}\| \cos\theta$

The score depends on two things:

1. **Direction:** How aligned are $\boldsymbol{k}_i$ and $\boldsymbol{q}$? ($\cos\theta$)
2. **Magnitude:** How large are $\|\boldsymbol{k}_i\|$ and $\|\boldsymbol{q}\|$?

Both matter! A key can get a high score by being well-aligned with the query, *or* simply by having a large norm.

Copying in Attention

Question: If one key $\boldsymbol{k}_j$ is perfectly aligned with $\boldsymbol{q}$ and all other keys are orthogonal to $\boldsymbol{q}$, what does $\boldsymbol{c}$ look like?

Copying in Attention

Question: If one key $\boldsymbol{k}_j$ is perfectly aligned with $\boldsymbol{q}$ and all other keys are orthogonal to $\boldsymbol{q}$, what does $\boldsymbol{c}$ look like?

The aligned key gets score $\|\boldsymbol{k}_j\| \, \|\boldsymbol{q}\| > 0$. All others get score 0.

After softmax: $\alpha_j = \frac{e^{\|\boldsymbol{k}_j\| \|\boldsymbol{q}\|}}{e^{\|\boldsymbol{k}_j\| \|\boldsymbol{q}\|} + (n-1) \underbrace{e^0}_{=1}}$

Copying in Attention

Question: If one key $\boldsymbol{k}_j$ is perfectly aligned with $\boldsymbol{q}$ and all other keys are orthogonal to $\boldsymbol{q}$, what does $\boldsymbol{c}$ look like?

The aligned key gets score $\|\boldsymbol{k}_j\| \, \|\boldsymbol{q}\| > 0$. All others get score 0.

After softmax: $\alpha_j = \dfrac{e^{\|\boldsymbol{k}_j\| \|\boldsymbol{q}\|}}{e^{\|\boldsymbol{k}_j\| \|\boldsymbol{q}\|} + (n-1) \underbrace{e^0}_{=1}}$

For large $\|\boldsymbol{q}\|$, this approaches 1. So:

$$\boxed{\boldsymbol{c} \approx \boldsymbol{v}_j}$$

The attention mechanism copies value $\boldsymbol{v}_j$ to the output!

Follow-up: What if we want to attend to *two* keys equally? Can we do that?

Softmax Temperature: Controlling Sharpness

The attention scores $s_i = \boldsymbol{k}_i^\top \boldsymbol{q}$ go through softmax. What if we scale them?

$$\alpha_i = \frac{\exp(\beta\, s_i)}{\sum_j \exp(\beta\, s_j)}$$

Question: What happens as β gets very large? Very small?

Softmax Temperature: Controlling Sharpness

The attention scores $s_i = \boldsymbol{k}_i^\top \boldsymbol{q}$ go through softmax. What if we scale them?

$$\alpha_i = \frac{\exp(\beta\, s_i)}{\sum_j \exp(\beta\, s_j)}$$

Question: What happens as β gets very large? Very small?

- $\beta \to \infty$: distribution becomes peaked (approaches one-hot at the max)
- $\beta \to 0$: distribution becomes uniform ($\alpha_i \approx 1/n$)
- $\beta = 1$: standard softmax

Softmax Temperature: Controlling Sharpness

The attention scores $s_i = \boldsymbol{k}_i^\top \boldsymbol{q}$ go through softmax. What if we scale them?

$$\alpha_i = \frac{\exp(\beta\, s_i)}{\sum_j \exp(\beta\, s_j)}$$

Question: What happens as β gets very large? Very small?

- $\beta \to \infty$: distribution becomes peaked (approaches one-hot at the max)
- $\beta \to 0$: distribution becomes uniform ($\alpha_i \approx 1/n$)
- $\beta = 1$: standard softmax

Key observation: Multiplying $\boldsymbol{q}$ by a scalar β is the same as scaling all scores by β.
So the *norm* of the query controls how "decisive" the attention is.

Example: Scores $[1, 1, 0]$ at Different Temperatures

Suppose three keys have scores $s_1 = 1, s_2 = 1, s_3 = 0$.

Question: What are the attention weights at $\beta = 1$?

Example: Scores $[1, 1, 0]$ at Different Temperatures

Suppose three keys have scores $s_1 = 1, s_2 = 1, s_3 = 0$.

Question: What are the attention weights at $\beta = 1$?

β	α_1	α_2	α_3
1	0.42	0.42	0.16

Example: Scores $[1, 1, 0]$ at Different Temperatures

Suppose three keys have scores $s_1 = 1, s_2 = 1, s_3 = 0$.

Question: What are the attention weights at $\beta = 1$?

β	α_1	α_2	α_3
1	0.42	0.42	0.16
10	≈ 0.50	≈ 0.50	≈ 0
100	≈ 0.50	≈ 0.50	≈ 0

At $\beta = 1$: the third key (score 0) still gets 16% of the weight!
At $\beta = 10$: it's essentially ignored. The two tied leaders split evenly.

Example: Scores $[1, 1, 0]$ at Different Temperatures

Suppose three keys have scores $s_1 = 1, s_2 = 1, s_3 = 0$.

Question: What are the attention weights at $\beta = 1$?

β	α_1	α_2	α_3
1	0.42	0.42	0.16
10	≈ 0.50	≈ 0.50	≈ 0
100	≈ 0.50	≈ 0.50	≈ 0

At $\beta = 1$: the third key (score 0) still gets 16% of the weight!
At $\beta = 10$: it's essentially ignored. The two tied leaders split evenly.

Takeaway: You can make attention *selective* by scaling up the query. But even equal scores don't give exactly $\frac{1}{2}$ — the non-max entries always dilute.

Orthogonal Keys: A Clean Setting

A useful special case: keys are **orthonormal**.

- $\boldsymbol{k}_i^\top \boldsymbol{k}_j = 0$ for $i \neq j$ (orthogonal)
- $\|\boldsymbol{k}_i\| = 1$ (unit norm)

If you build $\boldsymbol{q}$ as a linear combination of some keys, the dot products are easy to compute.

Exercise: Let $\boldsymbol{q} = 3\boldsymbol{k}_1 + 3\boldsymbol{k}_2$, with 4 orthonormal keys.

What is $\boldsymbol{k}_1^\top \boldsymbol{q}$? What is $\boldsymbol{k}_3^\top \boldsymbol{q}$?

Orthogonal Keys: A Clean Setting

A useful special case: keys are **orthonormal**.

- $\boldsymbol{k}_i^\top \boldsymbol{k}_j = 0$ for $i \neq j$ (orthogonal)
- $\|\boldsymbol{k}_i\| = 1$ (unit norm)

If you build $\boldsymbol{q}$ as a linear combination of some keys, the dot products are easy to compute.

Exercise: Let $\boldsymbol{q} = 3\boldsymbol{k}_1 + 3\boldsymbol{k}_2$, with 4 orthonormal keys.

What is $\boldsymbol{k}_1^\top \boldsymbol{q}$? What is $\boldsymbol{k}_3^\top \boldsymbol{q}$?

$$\boldsymbol{k}_1^\top \boldsymbol{q} = 3\underbrace{\boldsymbol{k}_1^\top \boldsymbol{k}_1}_{=1} + 3\underbrace{\boldsymbol{k}_1^\top \boldsymbol{k}_2}_{=0} = 3 \qquad \boldsymbol{k}_3^\top \boldsymbol{q} = 3\underbrace{\boldsymbol{k}_3^\top \boldsymbol{k}_1}_{=0} + 3\underbrace{\boldsymbol{k}_3^\top \boldsymbol{k}_2}_{=0} = 0$$

Orthogonal Keys: Attention Weights

So the scores are $[3, 3, 0, 0]$.

Question: After softmax, do keys 1 and 2 get exactly $\frac{1}{2}$ each?

Orthogonal Keys: Attention Weights

So the scores are $[3, 3, 0, 0]$.

Question: After softmax, do keys 1 and 2 get exactly $\frac{1}{2}$ each?

No! $\alpha_1 = \alpha_2 = \frac{e^3}{2e^3+2e^0} = \frac{20.09}{2(20.09)+2} \approx 0.475$.
Keys 3 and 4 still get ≈ 0.025 each. Not negligible!

Question: How could we make keys 1 and 2 dominate more?

Orthogonal Keys: Attention Weights

So the scores are $[3, 3, 0, 0]$.

Question: After softmax, do keys 1 and 2 get exactly $\frac{1}{2}$ each?

No! $\alpha_1 = \alpha_2 = \frac{e^3}{2e^3+2e^0} = \frac{20.09}{2(20.09)+2} \approx 0.475$.
Keys 3 and 4 still get ≈ 0.025 each. Not negligible!

Question: How could we make keys 1 and 2 dominate more?

Scale up! Use $\boldsymbol{q} = 30(\boldsymbol{k}_1 + \boldsymbol{k}_2)$ instead of $3(\boldsymbol{k}_1 + \boldsymbol{k}_2)$. Scores become $[30, 30, 0, 0]$, and the softmax concentrates almost entirely on keys 1 and 2.

Takeaway: With orthonormal keys, linear combinations give precise control over scores, and scaling controls selectivity.

What Happens When Keys Are Noisy?

In practice, keys are computed by a neural network — they're noisy.

Model this as: $\boldsymbol{k}_i \sim \mathcal{N}(\boldsymbol{\mu}_i, \Sigma_i)$, where $\boldsymbol{\mu}_i$ are orthonormal means.

Two noise models:

1. **Isotropic:** $\Sigma_i = \alpha \mathbf{I}$ (tiny, equal noise in all directions).
   Keys stay close to their means.
2. **Anisotropic:** $\Sigma_a = \alpha \mathbf{I} + \frac{1}{2}\boldsymbol{\mu}_a \boldsymbol{\mu}_a^\top$ (large noise along $\boldsymbol{\mu}_a$).
   Key $\boldsymbol{k}_a$ points in roughly the right direction, but its *magnitude* varies a lot.

Question: Under isotropic noise, does your carefully designed query still work? Why?

What Happens When Keys Are Noisy?

In practice, keys are computed by a neural network — they're noisy.

Model this as: $\boldsymbol{k}_i \sim \mathcal{N}(\boldsymbol{\mu}_i, \Sigma_i)$, where $\boldsymbol{\mu}_i$ are orthonormal means.

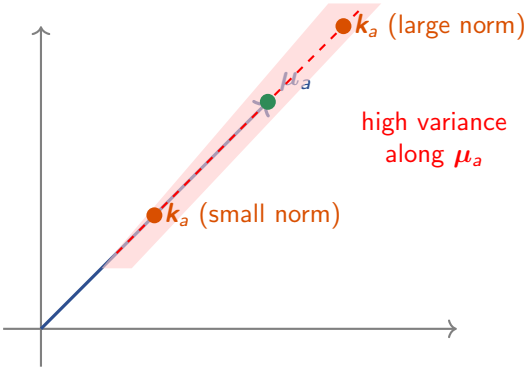
Two noise models:

1. **Isotropic:** $\Sigma_i = \alpha \mathbf{I}$ (tiny, equal noise in all directions).
   Keys stay close to their means.
2. **Anisotropic:** $\Sigma_a = \alpha \mathbf{I} + \frac{1}{2}\boldsymbol{\mu}_a \boldsymbol{\mu}_a^\top$ (large noise along $\boldsymbol{\mu}_a$).
   Key $\boldsymbol{k}_a$ points in roughly the right direction, but its *magnitude* varies a lot.

Question: Under isotropic noise, does your carefully designed query still work? Why?

Yes — tiny noise in all directions barely changes any dot product. The scores stay close to their noiseless values.

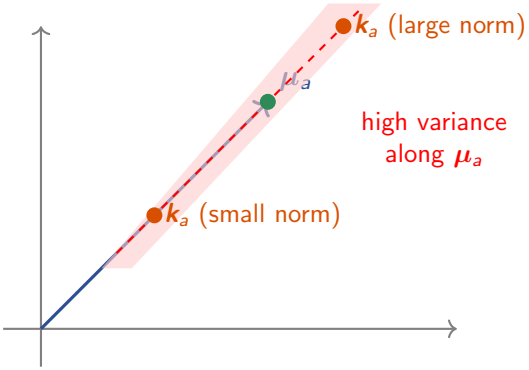
Visualizing Anisotropic Noise



Direction is roughly preserved, but the magnitude fluctuates.

Question: If your query has a component along μ_a, what happens to $k_a^\top q$ across different samples?

Visualizing Anisotropic Noise



Direction is roughly preserved, but the magnitude fluctuates.

Question: If your query has a component along μ_a, what happens to $k_a^\top q$ across different samples?

It fluctuates a lot — the score is proportional to $\|k_a\|$, which has high variance. So α_a and therefore c become unstable.

Noise Direction Matters!

We said $\boldsymbol{k}_a = \boldsymbol{\mu}_a + \boldsymbol{\epsilon}$, so $\boldsymbol{k}_a^\top \boldsymbol{q} = \boldsymbol{\mu}_a^\top \boldsymbol{q} + \boldsymbol{\epsilon}^\top \boldsymbol{q}$.

The noise contribution to the score is $\boldsymbol{\epsilon}^\top \boldsymbol{q}$.

Question: When does this noise term cause big problems? When is it harmless?

Noise Direction Matters!

We said $\boldsymbol{k}_a = \boldsymbol{\mu}_a + \boldsymbol{\epsilon}$, so $\boldsymbol{k}_a^\top \boldsymbol{q} = \boldsymbol{\mu}_a^\top \boldsymbol{q} + \boldsymbol{\epsilon}^\top \boldsymbol{q}$.

The noise contribution to the score is $\boldsymbol{\epsilon}^\top \boldsymbol{q}$.

Question: When does this noise term cause big problems? When is it harmless?

- **Big problems:** when $\boldsymbol{\epsilon}$ is aligned with $\boldsymbol{q}$ — then $\boldsymbol{\epsilon}^\top \boldsymbol{q}$ is large and fluctuating.
- **Harmless:** when $\boldsymbol{\epsilon}$ is orthogonal to $\boldsymbol{q}$ — then $\boldsymbol{\epsilon}^\top \boldsymbol{q} \approx 0$ regardless of $\|\boldsymbol{\epsilon}\|$.

Noise Direction Matters!

We said $\boldsymbol{k}_a = \boldsymbol{\mu}_a + \boldsymbol{\epsilon}$, so $\boldsymbol{k}_a^\top \boldsymbol{q} = \boldsymbol{\mu}_a^\top \boldsymbol{q} + \boldsymbol{\epsilon}^\top \boldsymbol{q}$.

The noise contribution to the score is $\boldsymbol{\epsilon}^\top \boldsymbol{q}$.

Question: When does this noise term cause big problems? When is it harmless?

- **Big problems:** when $\boldsymbol{\epsilon}$ is aligned with $\boldsymbol{q}$ — then $\boldsymbol{\epsilon}^\top \boldsymbol{q}$ is large and fluctuating.
- **Harmless:** when $\boldsymbol{\epsilon}$ is orthogonal to $\boldsymbol{q}$ — then $\boldsymbol{\epsilon}^\top \boldsymbol{q} \approx 0$ regardless of $\|\boldsymbol{\epsilon}\|$.

The punchline: It's not just about *how much* noise there is, but *what direction* it's in relative to the query.

This will be important when we compare single-head vs. multi-head attention.

Multi-Head Attention

Why one head isn't enough

The Problem With a Single Query

Suppose we want $\boldsymbol{c} \approx \frac{1}{2}(\boldsymbol{v}_a + \boldsymbol{v}_b)$ — an equal mix of two values.

With **one query**, you need to attend to *both* keys simultaneously.

This means $\boldsymbol{q}$ must have components along both $\boldsymbol{\mu}_a$ and $\boldsymbol{\mu}_b$.

Question: If $\boldsymbol{k}_a$ has high variance along $\boldsymbol{\mu}_a$, and your single query $\boldsymbol{q}$ has a component along $\boldsymbol{\mu}_a$ (which it must, to attend to a), what goes wrong?

The Problem With a Single Query

Suppose we want $\boldsymbol{c} \approx \frac{1}{2}(\boldsymbol{v}_a + \boldsymbol{v}_b)$ — an equal mix of two values.

With **one query**, you need to attend to *both* keys simultaneously.

This means $\boldsymbol{q}$ must have components along both $\boldsymbol{\mu}_a$ and $\boldsymbol{\mu}_b$.

Question: If $\boldsymbol{k}_a$ has high variance along $\boldsymbol{\mu}_a$, and your single query $\boldsymbol{q}$ has a component along $\boldsymbol{\mu}_a$ (which it must, to attend to a), what goes wrong?

The score $\boldsymbol{k}_a^\top \boldsymbol{q}$ fluctuates wildly. Sometimes $\alpha_a \gg \alpha_b$ (over-attend to a), sometimes $\alpha_a \ll \alpha_b$ (under-attend to a). The output $\boldsymbol{c}$ has high variance — not reliably $\frac{1}{2}(\boldsymbol{v}_a + \boldsymbol{v}_b)$.

Multi-Head Attention: Split the Work

Multi-head (simplified, 2 heads):

- Head 1 uses query $\boldsymbol{q}_1 \rightarrow$ produces $\boldsymbol{c}_1$
- Head 2 uses query $\boldsymbol{q}_2 \rightarrow$ produces $\boldsymbol{c}_2$
- Final output: $\boldsymbol{c} = \frac{1}{2}(\boldsymbol{c}_1 + \boldsymbol{c}_2)$

Key idea: Each head can *specialize*.

Head 1 focuses on $\boldsymbol{v}_a$ alone. Head 2 focuses on $\boldsymbol{v}_b$ alone.

Question: If $\boldsymbol{k}_a$ is noisy along $\boldsymbol{\mu}_a$, which head is affected? Which head is safe?

Multi-Head Attention: Split the Work

Multi-head (simplified, 2 heads):

- Head 1 uses query $\boldsymbol{q}_1 \rightarrow$ produces $\boldsymbol{c}_1$
- Head 2 uses query $\boldsymbol{q}_2 \rightarrow$ produces $\boldsymbol{c}_2$
- Final output: $\boldsymbol{c} = \frac{1}{2}(\boldsymbol{c}_1 + \boldsymbol{c}_2)$

Key idea: Each head can *specialize*.

Head 1 focuses on $\boldsymbol{v}_a$ alone. Head 2 focuses on $\boldsymbol{v}_b$ alone.

Question: If $\boldsymbol{k}_a$ is noisy along $\boldsymbol{\mu}_a$, which head is affected? Which head is safe?

- Head 1 (query aligned with $\boldsymbol{\mu}_a$, attending to a): affected
- Head 2 (query aligned with $\boldsymbol{\mu}_b$, attending to b): safe — the noise is orthogonal to its query!

Multi-Head Attention: Split the Work

Multi-head (simplified, 2 heads):

- Head 1 uses query $\boldsymbol{q}_1 \rightarrow$ produces $\boldsymbol{c}_1$
- Head 2 uses query $\boldsymbol{q}_2 \rightarrow$ produces $\boldsymbol{c}_2$
- Final output: $\boldsymbol{c} = \frac{1}{2}(\boldsymbol{c}_1 + \boldsymbol{c}_2)$

Key idea: Each head can *specialize*.

Head 1 focuses on $\boldsymbol{v}_a$ alone. Head 2 focuses on $\boldsymbol{v}_b$ alone.

Question: If $\boldsymbol{k}_a$ is noisy along $\boldsymbol{\mu}_a$, which head is affected? Which head is safe?

- Head 1 (query aligned with $\boldsymbol{\mu}_a$, attending to a): affected
- Head 2 (query aligned with $\boldsymbol{\mu}_b$, attending to b): safe — the noise is orthogonal to its query!

Only half the output is noisy $\Rightarrow$ lower overall variance.

Why Is Head 2 Safe? A Closer Look

$\boldsymbol{k}_a$'s noise is along $\boldsymbol{\mu}_a$. Head 2's query is along $\boldsymbol{\mu}_b$.

Question: What is $\boldsymbol{k}_a^\top \boldsymbol{q}_2$ when the noise is along $\boldsymbol{\mu}_a$ and $\boldsymbol{q}_2$ is along $\boldsymbol{\mu}_b$?

Why Is Head 2 Safe? A Closer Look

$\boldsymbol{k}_a$'s noise is along $\boldsymbol{\mu}_a$. Head 2's query is along $\boldsymbol{\mu}_b$.

Question: What is $\boldsymbol{k}_a^\top \boldsymbol{q}_2$ when the noise is along $\boldsymbol{\mu}_a$ and $\boldsymbol{q}_2$ is along $\boldsymbol{\mu}_b$?

$$\boldsymbol{k}_a^\top \boldsymbol{q}_2 = (\boldsymbol{\mu}_a + \boldsymbol{\epsilon})^\top \boldsymbol{q}_2 = \underbrace{\boldsymbol{\mu}_a^\top \boldsymbol{q}_2}_{\approx 0 \text{ (orthogonal)}} + \underbrace{\boldsymbol{\epsilon}^\top \boldsymbol{q}_2}_{\approx 0 \text{ (noise } \perp \text{ query)}}$$

The noisy key $\boldsymbol{k}_a$ gets a near-zero score in head 2, regardless of its norm. Head 2 is insulated from the noise.

Question: If we used the same query for both heads ($\boldsymbol{q}_1 = \boldsymbol{q}_2$), would we still get this benefit?

Why Is Head 2 Safe? A Closer Look

$\boldsymbol{k}_a$'s noise is along $\boldsymbol{\mu}_a$. Head 2's query is along $\boldsymbol{\mu}_b$.

Question: What is $\boldsymbol{k}_a^\top \boldsymbol{q}_2$ when the noise is along $\boldsymbol{\mu}_a$ and $\boldsymbol{q}_2$ is along $\boldsymbol{\mu}_b$?

$$\boldsymbol{k}_a^\top \boldsymbol{q}_2 = (\boldsymbol{\mu}_a + \boldsymbol{\epsilon})^\top \boldsymbol{q}_2 = \underbrace{\boldsymbol{\mu}_a^\top \boldsymbol{q}_2}_{\approx 0 \text{ (orthogonal)}} + \underbrace{\boldsymbol{\epsilon}^\top \boldsymbol{q}_2}_{\approx 0 \text{ (noise } \perp \text{ query)}}$$

The noisy key $\boldsymbol{k}_a$ gets a near-zero score in head 2, regardless of its norm. Head 2 is insulated from the noise.

Question: If we used the same query for both heads ($\boldsymbol{q}_1 = \boldsymbol{q}_2$), would we still get this benefit?

No! Both heads would suffer from the same noise. You'd just be averaging two copies of the same noisy output. The benefit comes from using *different* queries.

Positional Encodings

Why word order matters

The Transformer's Symmetry Problem

Consider the Transformer's two core operations:

1. **Self-attention:** Computes $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ from input $\mathbf{X}$, applies softmax.
2. **Feed-forward:** Applied independently to each position (row of $\mathbf{H}$).

Thought experiment: What happens if we shuffle the input tokens?

Represent shuffling as $\mathbf{X}_{\text{perm}} = \mathbf{P}\mathbf{X}$, where $\mathbf{P}$ is a permutation matrix.

Question: Compute $\mathbf{Q}_{\text{perm}} = \mathbf{X}_{\text{perm}}\mathbf{W}_Q$. Can you factor out $\mathbf{P}$?

The Transformer's Symmetry Problem

Consider the Transformer's two core operations:

1. **Self-attention:** Computes $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ from input $\mathbf{X}$, applies softmax.
2. **Feed-forward:** Applied independently to each position (row of $\mathbf{H}$).

Thought experiment: What happens if we shuffle the input tokens?

Represent shuffling as $\mathbf{X}_{\text{perm}} = \mathbf{P}\mathbf{X}$, where $\mathbf{P}$ is a permutation matrix.

Question: Compute $\mathbf{Q}_{\text{perm}} = \mathbf{X}_{\text{perm}}\mathbf{W}_Q$. Can you factor out $\mathbf{P}$?

$\mathbf{Q}_{\text{perm}} = \mathbf{P}\mathbf{X}\mathbf{W}_Q = \mathbf{P}\mathbf{Q}$. Yes — $\mathbf{P}$ just passes through the linear projection.

Same for $\mathbf{K}$ and $\mathbf{V}$: $\mathbf{K}_{\text{perm}} = \mathbf{P}\mathbf{K}$, $\mathbf{V}_{\text{perm}} = \mathbf{P}\mathbf{V}$.

Chasing $\mathbf{P}$ Through Attention

We have $\mathbf{Q}_p = \mathbf{PQ}$, $\mathbf{K}_p = \mathbf{PK}$, $\mathbf{V}_p = \mathbf{PV}$.

Question: What is $\mathbf{Q}_p\mathbf{K}_p^\top$? (Hint: what is $(\mathbf{PK})^\top$?)

Chasing $\mathbf{P}$ Through Attention

We have $\mathbf{Q}_p = \mathbf{PQ}$, $\mathbf{K}_p = \mathbf{PK}$, $\mathbf{V}_p = \mathbf{PV}$.

Question: What is $\mathbf{Q}_p\mathbf{K}_p^\top$? (Hint: what is $(\mathbf{PK})^\top$?)

$(\mathbf{PK})^\top = \mathbf{K}^\top\mathbf{P}^\top$, so $\mathbf{Q}_p\mathbf{K}_p^\top = \mathbf{PQK}^\top\mathbf{P}^\top$.

Question: After applying softmax (given: $\text{softmax}(\mathbf{PAP}^\top) = \mathbf{P}\,\text{softmax}(\mathbf{A})\,\mathbf{P}^\top$) and multiplying by $\mathbf{V}_p = \mathbf{PV}$, what simplifies?

Chasing $\mathbf{P}$ Through Attention

We have $\mathbf{Q}_p = \mathbf{PQ}$, $\mathbf{K}_p = \mathbf{PK}$, $\mathbf{V}_p = \mathbf{PV}$.

Question: What is $\mathbf{Q}_p\mathbf{K}_p^\top$? (Hint: what is $(\mathbf{PK})^\top$?)

$(\mathbf{PK})^\top = \mathbf{K}^\top\mathbf{P}^\top$, so $\mathbf{Q}_p\mathbf{K}_p^\top = \mathbf{PQK}^\top\mathbf{P}^\top$.

Question: After applying softmax (given: $\text{softmax}(\mathbf{PAP}^\top) = \mathbf{P}\,\text{softmax}(\mathbf{A})\,\mathbf{P}^\top$) and multiplying by $\mathbf{V}_p = \mathbf{PV}$, what simplifies?

$\mathbf{H}_p = \mathbf{P}\,\text{softmax}(\cdot)\,\underbrace{\mathbf{P}^\top\mathbf{P}}_{=\mathbf{I}}\,\mathbf{V} = \mathbf{PH}$.

The $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$ cancellation is the key step!

Permutation Equivariance

Tracing through the full model (attention + feed-forward):

$$\mathbf{Z}_{\text{perm}} = \mathbf{PZ}$$

Permutation Equivariance

Tracing through the full model (attention + feed-forward):

$$\mathbf{Z}_{\text{perm}} = \mathbf{PZ}$$

Shuffling the input just shuffles the output. Each token's representation is the same regardless of position.

Question: Why is this a problem for language? Can you think of a concrete example?

Permutation Equivariance

Tracing through the full model (attention + feed-forward):

$$\mathbf{Z}_{\text{perm}} = \mathbf{PZ}$$

Shuffling the input just shuffles the output. Each token's representation is the same regardless of position.

Question: Why is this a problem for language? Can you think of a concrete example?

"The dog bit the man" and "The man bit the dog" contain the same tokens. A permutation-equivariant model produces the same representations (just reordered) for both — it can't distinguish their meanings.

Question: How could we break this symmetry?

Permutation Equivariance

Tracing through the full model (attention + feed-forward):

$$\mathbf{Z}_{\text{perm}} = \mathbf{PZ}$$

Shuffling the input just shuffles the output. Each token's representation is the same regardless of position.

Question: Why is this a problem for language? Can you think of a concrete example?

"The dog bit the man" and "The man bit the dog" contain the same tokens. A permutation-equivariant model produces the same representations (just reordered) for both — it can't distinguish their meanings.

Question: How could we break this symmetry?

Add position-dependent information to the input! If each position has a unique "stamp," shuffling changes what the model sees.

Permutation Matrices: Quick Refresher

A permutation matrix $\mathbf{P} \in \mathbb{R}^{T \times T}$ reorders rows:

$$\mathbf{P} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \quad \Rightarrow \quad \mathbf{PX} = \begin{pmatrix} \text{row 2 of } \mathbf{X} \\ \text{row 3 of } \mathbf{X} \\ \text{row 1 of } \mathbf{X} \end{pmatrix}$$

Key properties:

- $\mathbf{P}$ is orthogonal: $\mathbf{P}^\top \mathbf{P} = \mathbf{P}\mathbf{P}^\top = \mathbf{I}$
- $\text{softmax}(\mathbf{P}\mathbf{A}\mathbf{P}^\top) = \mathbf{P}\,\text{softmax}(\mathbf{A})\,\mathbf{P}^\top$
- $\text{ReLU}(\mathbf{PA}) = \mathbf{P}\,\text{ReLU}(\mathbf{A})$

Question: If $\mathbf{1}$ is a column vector of all ones, what is $\mathbf{P1}$?

Permutation Matrices: Quick Refresher

A permutation matrix $\mathbf{P} \in \mathbb{R}^{T \times T}$ reorders rows:

$$\mathbf{P} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \quad \Rightarrow \quad \mathbf{PX} = \begin{pmatrix} \text{row 2 of } \mathbf{X} \\ \text{row 3 of } \mathbf{X} \\ \text{row 1 of } \mathbf{X} \end{pmatrix}$$

Key properties:

- $\mathbf{P}$ is orthogonal: $\mathbf{P}^\top \mathbf{P} = \mathbf{P}\mathbf{P}^\top = \mathbf{I}$
- $\text{softmax}(\mathbf{P}\mathbf{A}\mathbf{P}^\top) = \mathbf{P}\,\text{softmax}(\mathbf{A})\,\mathbf{P}^\top$
- $\text{ReLU}(\mathbf{P}\mathbf{A}) = \mathbf{P}\,\text{ReLU}(\mathbf{A})$

Question: If $\mathbf{1}$ is a column vector of all ones, what is $\mathbf{P}\mathbf{1}$?

$\mathbf{P}\mathbf{1} = \mathbf{1}$. Permuting a vector of all ones just gives all ones again. (This comes up when tracing $\mathbf{P}$ through the feed-forward bias terms.)

Sinusoidal Positional Encodings

Solution: Add a position-dependent vector to each token embedding.

$$\mathbf{X}_{\text{pos}} = \mathbf{X} + \Phi$$

where $\Phi \in \mathbb{R}^{T \times d}$ encodes positions using sines and cosines:

$$\Phi(t, 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right), \qquad \Phi(t, 2i{+}1) = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

Each dimension oscillates at a **different frequency**:

- Low $i \Rightarrow$ high frequency (changes rapidly with position)
- High $i \Rightarrow$ low frequency (changes slowly)

The result: every position gets a unique "fingerprint" of sines and cosines.

Why Do Positional Encodings Fix the Problem?

Without Φ: permuting input $\mathbf{X}$ gives permuted output $\mathbf{PZ}$. No position info.

With Φ: the model sees $\mathbf{X} + \Phi$, not just $\mathbf{X}$.

Question: If we shuffle the tokens, the model receives $\mathbf{PX} + \Phi$. For the old equivariance to hold, we'd need $\mathbf{PX} + \mathbf{P}\Phi$. Are these the same?

Why Do Positional Encodings Fix the Problem?

Without Φ: permuting input $\mathbf{X}$ gives permuted output $\mathbf{PZ}$. No position info.

With Φ: the model sees $\mathbf{X} + \Phi$, not just $\mathbf{X}$.

Question: If we shuffle the tokens, the model receives $\mathbf{PX} + \Phi$. For the old equivariance to hold, we'd need $\mathbf{PX} + \mathbf{P}\Phi$. Are these the same?

No! $\mathbf{P}\Phi \neq \Phi$ because different positions have different encodings. Permuting the position encodings changes them, but the model always uses the *fixed* Φ. So the model sees genuinely different inputs for different orderings — symmetry is broken.

Question: Could two different positions $t_1 \neq t_2$ ever have the same encoding $\Phi(t_1, :) = \Phi(t_2, :)$?

Why Do Positional Encodings Fix the Problem?

Without Φ: permuting input $\mathbf{X}$ gives permuted output $\mathbf{PZ}$. No position info.

With Φ: the model sees $\mathbf{X} + \Phi$, not just $\mathbf{X}$.

Question: If we shuffle the tokens, the model receives $\mathbf{PX} + \Phi$. For the old equivariance to hold, we'd need $\mathbf{PX} + \mathbf{P}\Phi$. Are these the same?

No! $\mathbf{P}\Phi \neq \Phi$ because different positions have different encodings. Permuting the position encodings changes them, but the model always uses the *fixed* Φ. So the model sees genuinely different inputs for different orderings — symmetry is broken.

Question: Could two different positions $t_1 \neq t_2$ ever have the same encoding $\Phi(t_1, :) = \Phi(t_2, :)$?

Not for practical sequence lengths. Individual dimensions repeat (sine is periodic), but the combination of all dimensions at different incommensurable frequencies doesn't. The lowest-frequency period is $\approx 62{,}832$ — far longer than any practical sequence.

Position Encoding: Intuition Check

Question: Suppose $d = 2$ (just one sine and one cosine at the same frequency). Could two positions then have the same encoding?

Position Encoding: Intuition Check

Question: Suppose $d = 2$ (just one sine and one cosine at the same frequency). Could two positions then have the same encoding?

Yes! With a single frequency ω, $\Phi(t) = (\sin(\omega t), \cos(\omega t))$. This repeats with period $2\pi/\omega$. Having *multiple* frequencies at incommensurable rates is what makes each position unique.

Question: Modern models (GPT-2, LLaMA) don't use sinusoidal encodings. What do they use instead?

Position Encoding: Intuition Check

Question: Suppose $d = 2$ (just one sine and one cosine at the same frequency). Could two positions then have the same encoding?

Yes! With a single frequency ω, $\Phi(t) = (\sin(\omega t), \cos(\omega t))$. This repeats with period $2\pi/\omega$. Having *multiple* frequencies at incommensurable rates is what makes each position unique.

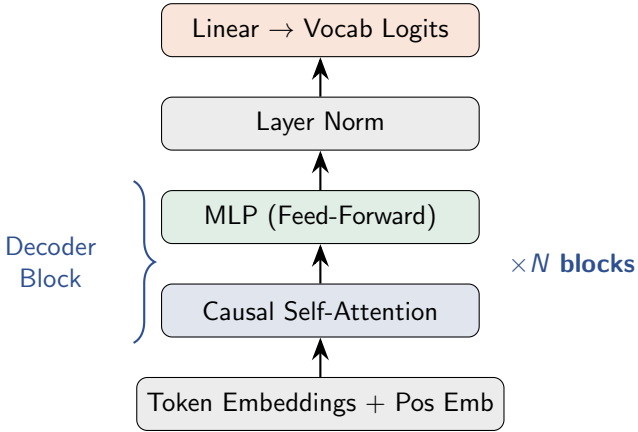
Question: Modern models (GPT-2, LLaMA) don't use sinusoidal encodings. What do they use instead?

GPT-2 uses *learned* position embeddings (a trainable lookup table). LLaMA and many recent models use RoPE (Rotary Position Embeddings). Sinusoidal encodings are a clean theoretical starting point, but learned/rotary approaches tend to work better in practice.

Decoder Architecture

Putting it all together

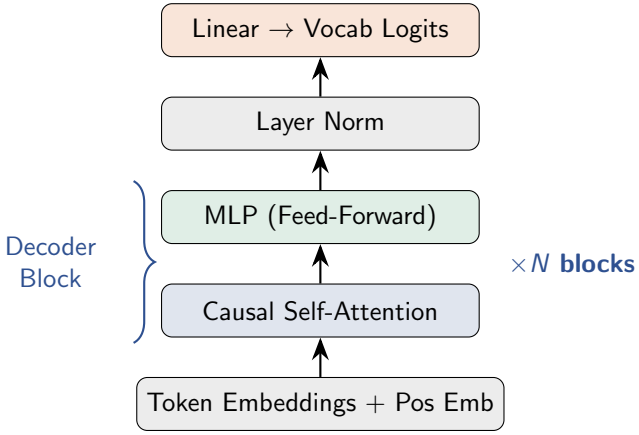
GPT-2 Style Decoder



Tokens come in at the bottom. N identical decoder blocks. Final layer norm + linear head to produce vocabulary logits.

Question: Where do positional encodings enter? Where does the causal mask get applied?

GPT-2 Style Decoder



Tokens come in at the bottom. N identical decoder blocks. Final layer norm + linear head to produce vocabulary logits.

Question: Where do positional encodings enter? Where does the causal mask get applied?

Pos. encodings: added at the very bottom (input). Causal mask: inside each self-attention.

Multi-Head Attention in Practice

Steps:

1. Project input into $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ via learned weight matrices
2. **Reshape** each into multiple heads: `(B, T, d)` $\rightarrow$ `(B, h, T, d/h)`
3. Compute attention *per head* (parallel via batched matmul)
4. Concatenate heads, project back to model dimension

Key shapes:

- After projection: `(batch, seq_len, d_model)`
- After multi-head reshape: `(batch, n_heads, seq_len, d_head)`
- Attention scores: `(batch, n_heads, seq_len, seq_len)`

Question: Why reshape instead of slicing? (e.g., why not take columns 0–63 for head 1, 64–127 for head 2?)

Multi-Head Attention in Practice

Steps:

1. Project input into $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ via learned weight matrices
2. **Reshape** each into multiple heads: `(B, T, d)` $\rightarrow$ `(B, h, T, d/h)`
3. Compute attention *per head* (parallel via batched matmul)
4. Concatenate heads, project back to model dimension

Key shapes:

- After projection: `(batch, seq_len, d_model)`
- After multi-head reshape: `(batch, n_heads, seq_len, d_head)`
- Attention scores: `(batch, n_heads, seq_len, seq_len)`

Question: Why reshape instead of slicing? (e.g., why not take columns 0–63 for head 1, 64–127 for head 2?)

After the linear projection, all input dimensions are mixed. Reshaping after projection is equivalent and lets you use a single batched matmul for all heads in parallel.

Causal Masking: No Peeking at the Future

In a **decoder**, token t can only attend to tokens $\leq t$.

Enforced by adding a mask *before* softmax:

$$\text{Attention} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_{\text{head}}}} + \text{mask}\right)\mathbf{V}$$

where:

$$\text{mask}_{ij} = \begin{cases} 0 & \text{if } j \leq i \ \text{ (allowed)} \\ -\infty & \text{if } j > i \ \text{ (blocked)} \end{cases}$$

After softmax, $e^{-\infty} = 0$, so future positions get zero weight.

Why Scale by $\sqrt{d_{\text{head}}}$?

Question: If $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{d_{\text{head}}}$ have entries $\sim \mathcal{N}(0, 1)$ independently, what is the variance of $\boldsymbol{q}^\top \boldsymbol{k}$?

Why Scale by $\sqrt{d_{\text{head}}}$?

Question: If $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{d_{\text{head}}}$ have entries $\sim \mathcal{N}(0,1)$ independently, what is the variance of $\boldsymbol{q}^\top \boldsymbol{k}$?

$\text{Var}(\boldsymbol{q}^\top \boldsymbol{k}) = d_{\text{head}}$. (It's a sum of d_{head} independent products, each with variance 1.)

Question: What goes wrong if scores have variance d_{head} and d_{head} is large (say 64)?

Why Scale by $\sqrt{d_{\text{head}}}$?

Question: If $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{d_{\text{head}}}$ have entries $\sim \mathcal{N}(0,1)$ independently, what is the variance of $\boldsymbol{q}^\top \boldsymbol{k}$?

$\text{Var}(\boldsymbol{q}^\top \boldsymbol{k}) = d_{\text{head}}$. (It's a sum of d_{head} independent products, each with variance 1.)

Question: What goes wrong if scores have variance d_{head} and d_{head} is large (say 64)?

Scores are large in magnitude $\Rightarrow$ softmax saturates to near-one-hot $\Rightarrow$ gradients are tiny $\Rightarrow$ training is very slow or stuck.

Why Scale by $\sqrt{d_{\text{head}}}$?

Question: If $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{d_{\text{head}}}$ have entries $\sim \mathcal{N}(0,1)$ independently, what is the variance of $\boldsymbol{q}^\top \boldsymbol{k}$?

$\text{Var}(\boldsymbol{q}^\top \boldsymbol{k}) = d_{\text{head}}$. (It's a sum of d_{head} independent products, each with variance 1.)

Question: What goes wrong if scores have variance d_{head} and d_{head} is large (say 64)?

Scores are large in magnitude $\Rightarrow$ softmax saturates to near-one-hot $\Rightarrow$ gradients are tiny $\Rightarrow$ training is very slow or stuck.

Dividing by $\sqrt{d_{\text{head}}}$ normalizes scores to have unit variance, keeping softmax in a well-behaved regime regardless of dimension.

Common bug: Dividing by $\sqrt{d_{\text{model}}}$ instead of $\sqrt{d_{\text{head}}}$. The dot product is per head!

Training: Next-Token Prediction

Language modeling objective:
Given tokens $[t_0, t_1, t_2, \ldots, t_T]$:

- **Input:** $[t_0, t_1, \ldots, t_{T-1}]$
- **Target:** $[t_1, t_2, \ldots, t_T]$ (shifted by one)

At each position, predict the *next* token. Loss = cross-entropy over vocabulary.

Greedy generation:

1. Start with a prompt
2. Forward pass $\rightarrow$ take logits at the last position
3. Argmax $\rightarrow$ predicted next token
4. Append to sequence, repeat

Question: What happens if the sequence grows longer than the model's context length?

Training: Next-Token Prediction

Language modeling objective:
Given tokens $[t_0, t_1, t_2, \ldots, t_T]$:

- **Input:** $[t_0, t_1, \ldots, t_{T-1}]$
- **Target:** $[t_1, t_2, \ldots, t_T]$ (shifted by one)

At each position, predict the *next* token. Loss = cross-entropy over vocabulary.

Greedy generation:

1. Start with a prompt
2. Forward pass $\rightarrow$ take logits at the last position
3. Argmax $\rightarrow$ predicted next token
4. Append to sequence, repeat

Question: What happens if the sequence grows longer than the model's context length?

Truncate from the left — keep only the most recent `context_length` tokens as input.

Common Implementation Pitfalls

Attention

- Forgetting $\sqrt{d_{\text{head}}}$ scaling
- Wrong reshape for multi-head split
- Mask applied *after* softmax
- Mask has wrong sign or shape

Training / Generation

- Off-by-one in input vs. target split
- Applying softmax before `cross_entropy` (double softmax!)
- Not truncating to context length
- Forgetting `torch.no_grad()`

General advice:

- Add `assert` statements to check tensor shapes
- Build bottom-up: MLP $\rightarrow$ Attention $\rightarrow$ Block $\rightarrow$ Model
- Test each piece before assembling the whole thing

Summary

Concepts

- Attention = weighted average via softmax over dot-product scores
- Scaling the query $\Leftrightarrow$ controlling softmax temperature
- Multi-head attention isolates noise across heads
- Without positional encodings, Transformers are permutation equivariant

Implementation

- Multi-head: reshape, don't slice
- Causal mask: $-\infty$ before softmax
- Scale by $\sqrt{d_{\text{head}}}$
- Next-token prediction: shift inputs and targets by one

Assignment 3 is on Moodle now — due April 17.

Everything we covered today is directly relevant. Start early!